Does Copilot Cowork really run several models in one session?

Findings, evidence and caveats from live testing in a Microsoft 365 tenant

Date 1 September 2026 **Tenant** Microsoft demo tenant **Repository** github.com/ITSpecialist111/Copilot-Cowork-Skills

Status Draft for review before publication

> **READ THIS FIRST**
>
> Cowork does genuinely dispatch sub-agents onto different models inside a single session, and two independent instruments agree on which models ran. Two published Microsoft statements are contradicted by what the tenant actually produces. Three claims are weaker than they look and are labelled as such — please check **Section 7** before posting. The screenshots have been redacted; **Section 8, A2** needs a decision from you.

1. What was being tested, and why

Cowork is agentic: it plans, dispatches sub-agents and works in parallel. The marketing claim is that it is multi-model. The question worth answering is whether those sub-agents genuinely execute on *different* models, or whether "multi-agent" is just one model wearing several hats.

This matters practically. If three "perspectives" are one model, agreement between them is nearly worthless — it is the same set of priors restated three times. If they are genuinely different models from different vendors, agreement is real corroboration. Everything downstream, including whether a multi-model protocol is worth paying for, rests on that distinction.

Method

1. Built the Fusion Harness as a Cowork custom skill so runs would be repeatable and structured.
2. Ran controlled sessions with the model picker on Auto and on explicit selections.
3. Captured Cowork's own undocumented platform API through browser network interception.
4. Independently corroborated against the Microsoft Purview unified audit log.
5. Put every conclusion past a separate quality-assurance agent, which rejected two of them.

> **INSTRUMENT NOTE**
>
> A model's own account of which model it is **is not evidence**. In testing, a sub-agent running on Fable 5 confidently identified itself as "Claude Sonnet 4.5" — right vendor, wrong model — and two others said "unknown". Every claim below rests on platform-written metadata, never on self-report.

2. Findings at a glance

#	Finding	Evidence	Strength
F1	Three different models ran as parallel sub-agents inside one Cowork session	Platform session record	PROVEN
F2	Purview's audit log *does* carry Cowork model and provider, contradicting the documentation	6 audit records	PROVEN
F3	The two instruments agree exactly on model and timing	API vs Purview	PROVEN
F4	Reasoning effort is settable per sub-agent, not just per session	Applied and honoured	PROVEN
F5	On Auto, the model did not change with task complexity	2 sessions, both systems	INDICATED
F6	A three-model run costs roughly an order of magnitude more than a normal turn	2 measured runs	INDICATED
F7	There is no per-response model badge in the Cowork interface	Direct observation	PROVEN
F8	Duplicate skills stop a skill triggering; deletion needs explicit approval	1 failure, 1 recovery	LIKELY

3. F1 — Three models, one session, in parallel

Cowork exposes an undocumented platform API that the web client calls for its own use. One endpoint returns a live snapshot of the current task, including a record of every sub-agent dispatched. For a single Fusion Harness run it returned:

```
completedSubagents: [
  { name: "general-purpose", model: "gpt-5.6-sol",     status: "completed", duration_ms: 48775 },
  { name: "general-purpose", model: "claude-fable-5",   status: "completed", duration_ms: 48388 },
  { name: "general-purpose", model: "claude-sonnet-5", status: "completed", duration_ms: 47588 }
]
```

Three models. Two vendors. Durations within 1.2 seconds of each other, which is what parallel dispatch looks like — sequential execution would have produced roughly 48, 97 and 145 seconds of elapsed time, not three near-identical figures.

Two structural consequences follow, and both matter more than the headline:

- **Independence is structural, not promised.** Each sub-agent runs in its own context. It is not possible for one slot to have seen another's answer, because there is no shared transcript to see it in.
- **The session picker does not constrain the slots.** The compose-box model picker governs only the coordinating conversation. Sub-agent dispatch carries its own model parameter, so a session pinned to one model can still fan out across three.

4. F2 and F3 — Purview corroborates, and contradicts the documentation

An API the vendor has not documented is a single instrument, and a single instrument is not proof. The obvious second source is the Microsoft Purview unified audit log — except that the current Microsoft Learn page says it will not help:

MICROSOFT LEARN — AUDIT-COPILOT

"Currently, Cowork doesn't show provider details." The same page lists `ModelName` as "*Not available in Microsoft 365 Copilot scenarios*". **Both statements are contradicted by real records in this tenant.**

A genuine Cowork `CopilotInteraction` record contains:

```
"AppHost": "cowork",
"AppIdentity": "Copilot.M365Copilot.CoworkChat",
"ModelTransparencyDetails": [
  { "ModelName": "GPT-5.6 Terra", "ModelProviderName": "OpenAI" }
]
```

Six records were returned for the test window. Read against the API's own readings, the two systems line up exactly:

Time (UTC)	Purview `ModelName`	Platform API `activeModel`	What was happening
09:33	Opus 5 / Anthropic	`opus-5`	Auto, trivial task
09:34	null	`opus-5`	Auto — attribution absent
09:37	Opus 5 / Anthropic	`opus-5`	Auto, complex task
09:39	Opus 5 / Anthropic	`opus-5`	Auto, control session
09:43	null	`opus-5`	Auto — attribution absent
09:51	GPT-5.6 Terra / OpenAI	`gpt-5.6-terra`	Picker switched explicitly

Two independent systems, one written by the runtime and one written by the compliance pipeline, agreeing on model, provider and minute. That is what makes F1 and F5 defensible rather than anecdotal.

The two `null` entries are consistent with Microsoft's own caveat that on Auto the model "might not be available for every request". They are a gap in the record, not a contradiction — but they are the reason pinning a model explicitly is what makes an audit trail reliably informative.

5. Evidence exhibits

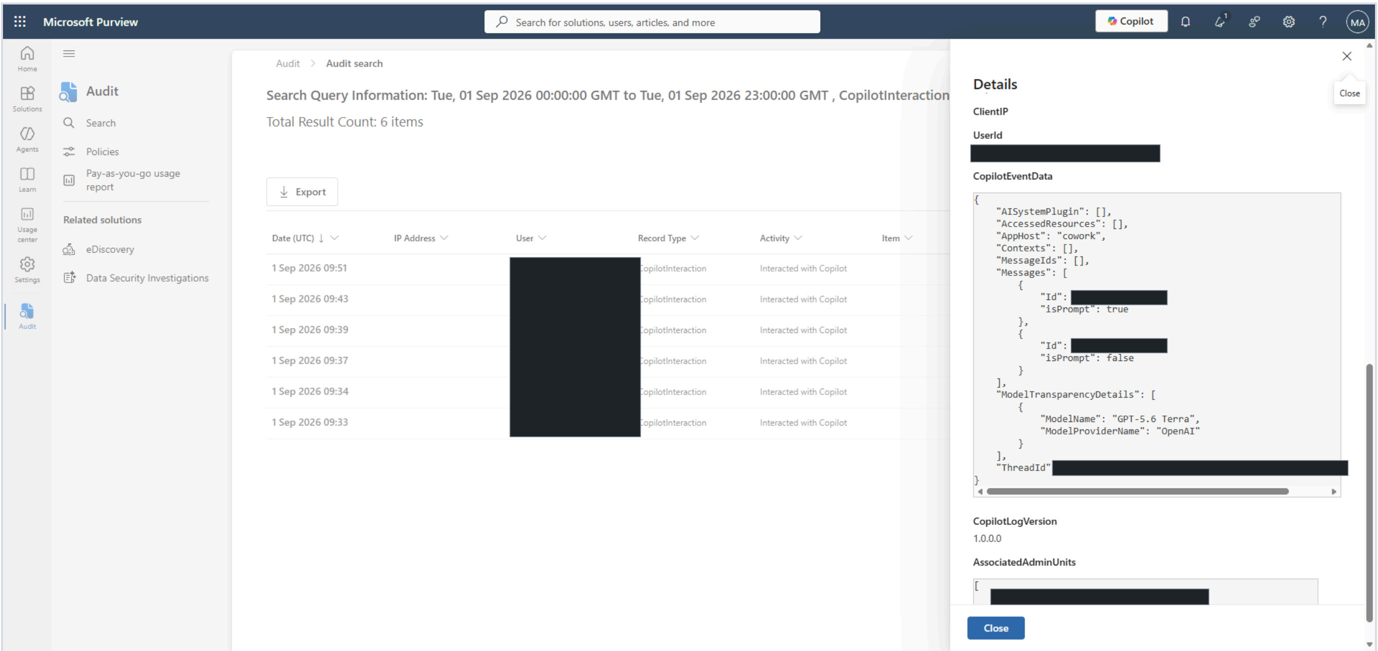

Exhibit 1 — Purview Audit search, six `CopilotInteraction` records for 1 September 2026. The Details pane shows `"AppHost": "cowork"` and `ModelTransparencyDetails` naming **GPT-5.6 Terra / OpenAI**. This is the record the documentation says does not exist. Tenant identifiers redacted.

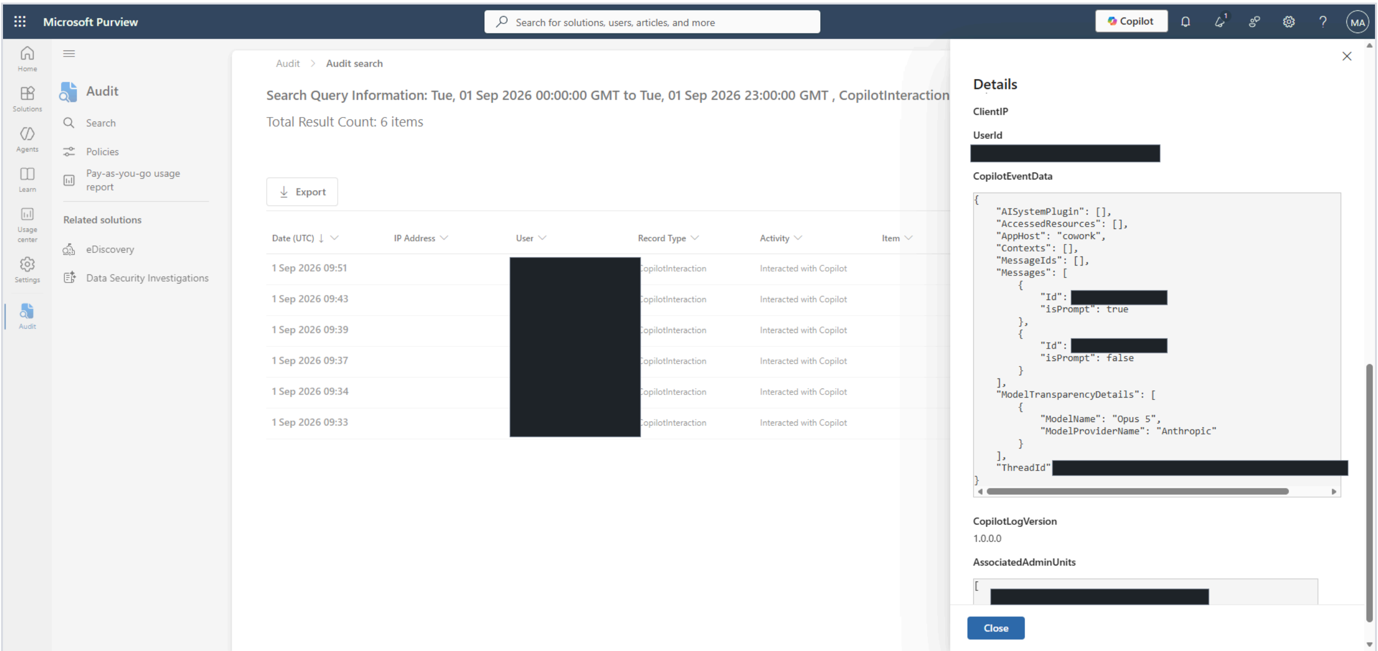

Exhibit 2 — The same search and the same six-record result set, a different interaction: **Opus 5 / Anthropic**. Two vendors attributed inside one audited window, with the query header visible in both shots so they cannot be mistaken for separate searches. Tenant identifiers redacted.

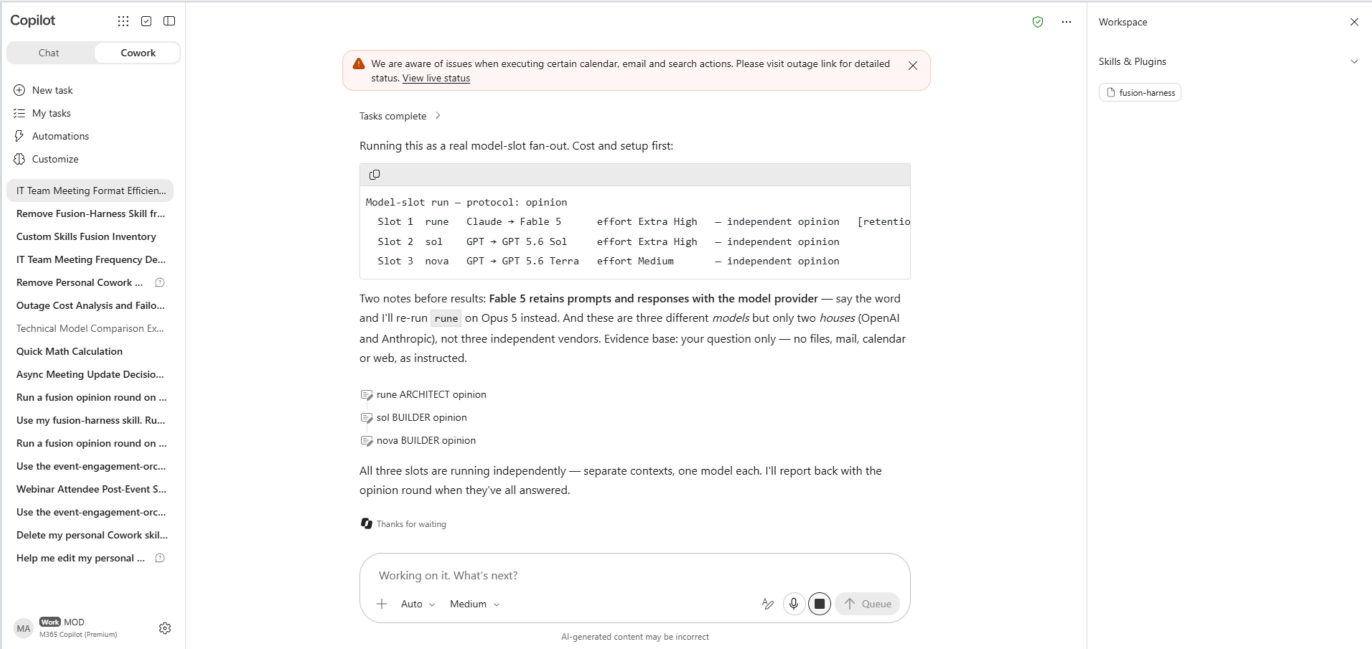

Exhibit 3 — Cowork loading the custom skill and printing the slot table before dispatching: rune on Fable 5 at Extra High, sol on GPT 5.6 Sol at Extra High, nova on GPT 5.6 Terra at Medium. Note it volunteers the Fable 5 data-retention warning and the caveat that three models across two vendors is not three independent vendors — both are contract behaviour from the skill, not improvisation.

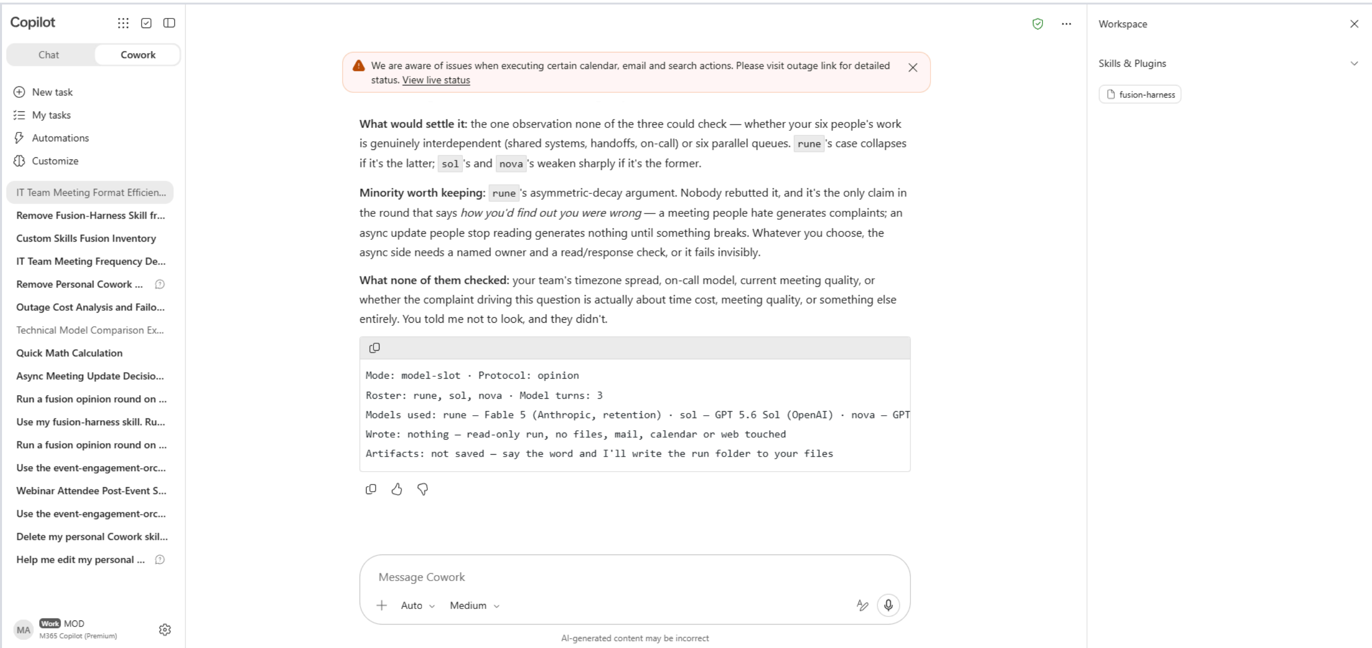

Exhibit 4 — The close-out ledger: what would settle the disagreement, the minority position worth keeping, what none of the models checked, and which models ran.

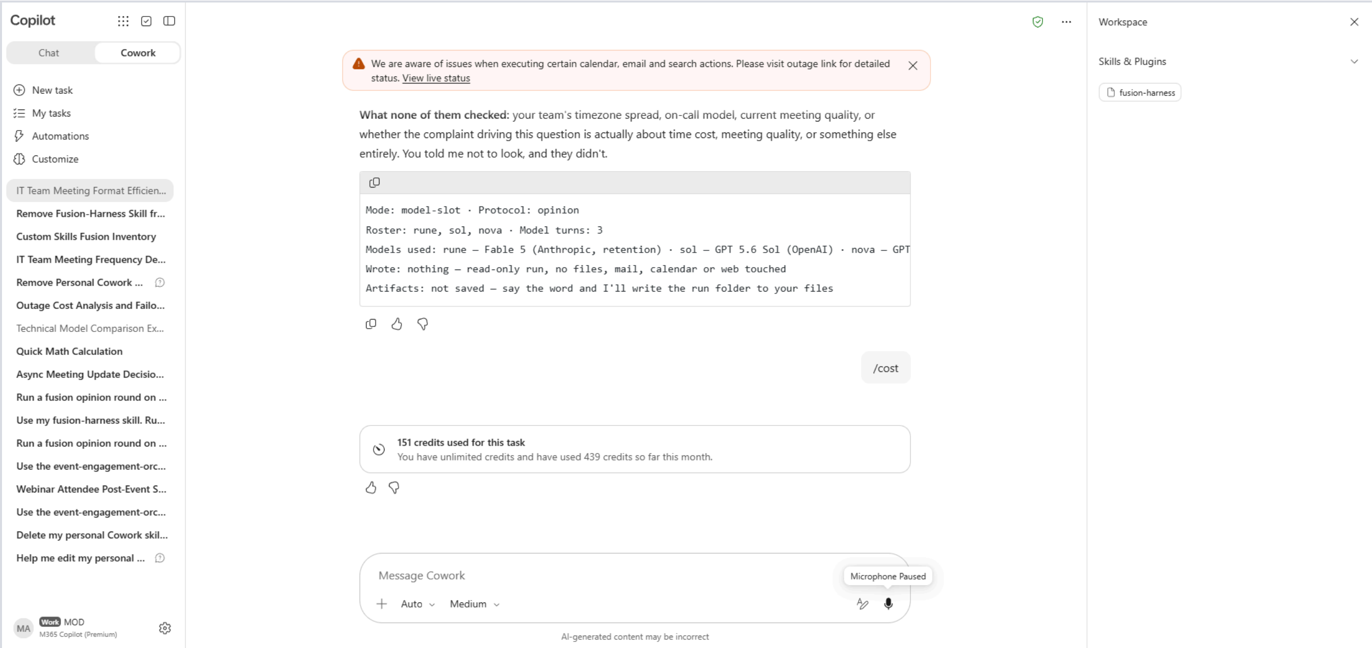

Exhibit 5 — `/cost` reporting **151 credits** for the three-model run. Cross-validated on another session against the platform API's own figure (`20` reported versus `20.5745` recorded).

6. What the skill adds on top

Cowork already has the orchestration. The Fusion Harness skill does not add sub-agents or parallelism — it adds the discipline that makes a multi-model run worth more than a single answer. Four things, each of which showed up unprompted in the verified run:

Mechanism	What it does, and why
Named slots on named models	An architect, a primary builder and a challenger, each pinned to a named model and effort level, dispatched in parallel. Personas alone would be three costumes on one model.
Independence you can check	Each slot answers in its own context and its answer is written to an artifact before anything is merged. The run is inspectable after the fact.
The divergence floor	When every slot agrees, the skill must say whether that agreement was *earned*. Verbatim from the run: "*Consensus — and it is earned... though note it's two houses, not three, so it isn't fully independent triangulation.*" Three agreeing perspectives are not three confirmations.
Cost on the record	Every run ends by asking the user to type `/cost`. The skill cannot invoke it — it is a client-side command — so stating a credit figure without it is prohibited.

What it cost

Two measured three-model fan-outs came in at **151** and **362 credits**. Ordinary single-turn sessions in the same tenant ran **11–38**. That spread is wide enough that the honest framing is an order of magnitude, not a price. The practical rule that follows: run the harness in its own fresh session, because `/cost` is scoped to a single task and mixing the run into an existing conversation makes its cost unrecoverable.

It is also the reason the skill's default mode is the cheap single-session one. An ensemble over a settled question produces three restatements and charges for three.

7. Limits — read before publishing

A separate QA agent reviewed the run and the write-up and rejected two conclusions outright before they reached this document. These are the claims that remain weaker than they may sound.

#	Do not say	Say instead
L1	"Auto always picks the top model regardless of task."	Two sessions is a small sample. Auto did not vary with complexity *in preliminary testing*, and two of six records carried no attribution at all. It cannot be excluded that Auto re-routes without writing `activeModel`.
L2	"A three-model run costs 151 credits."	151 and 362 across two runs. Debate and validation protocols cost more, and context size moves it. Quote it as approximate or not at all.
L3	"The duplicate skill caused the failure."	One failure before, one success after. Consistent with Microsoft's documented warning that two skills compete for the same requests, but n=1 either side. Likely, not proven.
L4	"Purview shows Cowork model attribution."	It does in this tenant, today. The docs say otherwise, so this reads as documentation lag. Tell readers to check their own tenant rather than asserting it about theirs.
L5	"The models were definitely the ones requested."	Attribution rests on platform metadata. If the runtime silently substituted a model while reporting the requested one, that is not detectable from outside.

> **HONEST DISCLOSURE ABOUT THE RUN IN EXHIBITS 4 AND 5**
>
> That run had two defects. It skipped writing artifacts, reasoning on its own that the question was conversational, and it dropped the cost line because it was the last line inside a fenced block. Both were faults in the skill file, both are now fixed — artifacts made mandatory with the judgement-call escape routes named and closed, and the cost request moved out of the fence into its own step. The screenshots are honest evidence of the run as it happened, not of the current skill. Worth saying so if anyone asks.

8. Before you post — actions

#	Priority	Action
A1	**Done**	**Exhibits 1 and 2 have been redacted.** Black boxes now cover the tenant admin UPN, the user id, the message ids, the Teams `ThreadId` and the admin unit GUID. The evidence that matters — `AppHost`, `ModelName`, `ModelProviderName`, the query header and the six-record count — is untouched and still legible.
A2	**Decision needed**	**The unredacted originals were pushed to the public repository** in commit `07b6977` before the redaction was done. Replacing the files in a new commit fixes what people see but leaves the originals reachable in git history. Purging them needs a history rewrite and a force push. Low severity — demo tenant, no credentials — but it is your call.
A3	Recommended	Keep the hedging in L1–L5 in the post. It is the difference between a credible finding and an overclaim someone can knock down in the comments.
A4	Recommended	Decide whether to name the undocumented platform API. It is the strongest evidence, but it is unsupported and may change without notice. The Purview route stands on its own and is supported.
A5	Optional	Attribute the upstream project. The post credits the original Fusion Harness by disler; the fork keeps full history so MIT attribution is preserved.

9. The draft post

Ready to paste. Roughly 480 words. Screenshots referenced by exhibit number — Exhibits 1 and 2 need the redaction in A1 first.

Copilot Cowork can run several models inside one session. I went looking for proof.

Cowork is agentic — it plans, dispatches sub-agents and works in parallel. What I wanted to know was whether those sub-agents genuinely run on *different* models, or whether that's just a description of one model doing several things.

They genuinely do. From Cowork's own session record, one task, three sub-agents:

```
model: gpt-5.6-sol      status: completed   duration: 48,775 ms
model: claude-fable-5   status: completed   duration: 48,388 ms
model: claude-sonnet-5  status: completed   duration: 47,588 ms
```

Three models, two vendors, near-identical durations — dispatched in parallel, inside a single session.

Then I checked the audit log, because a model's own account of itself is worthless. (Mine proved that nicely: a sub-agent running on Fable 5 confidently identified itself as "Claude Sonnet 4.5". Right vendor, wrong model.)

Microsoft Purview's unified audit log carries the answer. Every Cowork `CopilotInteraction` record includes:

```
"AppHost": "cowork",
"ModelTransparencyDetails": [
  { "ModelName": "GPT-5.6 Terra", "ModelProviderName": "OpenAI" }
]
```

That is platform-written, per interaction, and independent of anything the model says. Worth knowing: the current Microsoft Learn page states Cowork "doesn't show provider details" — in my tenant it plainly does. Documentation lag rather than a claim about your tenant, but check yours.

One thing that surprised me. With the picker on Auto, I ran a trivial task and a hard multi-step reasoning task in the same session. Both ran on Opus 5. A control session with the hard task first — also Opus 5. Change the picker explicitly, and the very next interaction switches model, confirmed in the audit log.

In preliminary testing (n=2 sessions) Auto's model assignment did not vary with task complexity. Explicit selection did. Small sample, and one interaction lacked model attribution in the audit record — but the pattern was consistent across both sessions.

Which is exactly why I built a skill for it.

The Fusion Harness is a Cowork skill that ports a multi-model protocol I'd been running outside M365. It doesn't add orchestration — Cowork already has that. What it adds is discipline:

- **Named slots on named models.** An architect, a builder, a challenger, each pinned to a different model and reasoning effort, dispatched in parallel.
- **Independence you can check.** Each slot answers in its own context and writes its own artifact before anything is merged.
- **Honesty about agreement.** If every slot agrees, the skill says so *and* says whether the agreement was earned. From my last run: "*Consensus — and it is earned… though note it's two houses, not three, so it isn't fully independent triangulation.*" Three agreeing perspectives across two vendors are not three confirmations.
- **Cost on the record.** Every run ends by asking for `/cost`. Two three-model fan-outs came in at 151 and 362 credits, against 11–38 for an ordinary session in the same tenant. Wide spread, so treat those as an order of

magnitude rather than a quote — but worth knowing before you make it a habit.

And a trap worth passing on. The skill stopped triggering entirely at one point. The cause wasn't the skill file — it was a *duplicate*. Deleting a skill in Cowork spawns an agentic task that stops at an approval gate, and mine sat unapproved while I uploaded the replacement. Two skills with near-identical descriptions, and neither fired. Delete, approve the deletion, confirm it's gone, *then* upload.

Ask Cowork a hard question and it will answer confidently. Ask three different models and make them show their working, and you find out which parts of that confidence were earned.

Built on the Fusion Harness by @disler. Testing done in a Microsoft demo tenant.
#MicrosoftCopilot #CopilotCowork #Microsoft365 #AgenticAI #Purview

---

Source files, the skill and the full evidence set: github.com/ITSpecialist111/Copilot-Cowork-Skills — `fusion-harness/`